

PROJECT REPORT: VIT BHOPAL LOST & FOUND SYSTEM

1. Cover Page Details

- **Project Title:** VIT Bhopal Lost & Found Campus Recovery Portal
- **Course Name:** Flipped Course Evaluation Project
- **Institution:** VIT Bhopal University
- **Technology Domain:** Full Stack Web Development and Applied Artificial Intelligence
- **Author / Student Name:** [Insert Your Name]
- **Registration Number:** [Insert Your Registration Number]
- **Submission Date:** September 2026

2. Introduction

- Contextualizes the challenges of managing personal belongings across academic and residential premises at VIT Bhopal.
- Introduces a centralized, responsive web platform built using Python, SQLite, HTML, CSS, and modern generative AI models.
- Bridges the operational gap caused by the lack of a physical lost and found department on the university campus.
- Establishes a verified, community-driven workflow restricted to campus affiliates to safely record, browse, and recover lost property.

3. Problem Statement

- Misplacement of high-value personal objects like scientific calculators, laptops, hostel keys, and institutional ID cards is a routine issue across campus spaces.
- The absence of a formal, physical recovery counter leaves finders and losers dependent on unstructured communication mediums.
- Current reporting via informal messaging groups on Telegram and WhatsApp suffers from rapid information decay, lack of verification, message clutter, and zero searchability.
- There is no authenticated, central repository that allows students to easily publish discovery details or track misplaced belongings.

4. Functional Requirements

- **Module 1: Domain-Gated Authentication & User Management**
 - Restricts account creation and login operations strictly to email identifiers containing the institutional suffix @vitbhopal.ac.in.
 - Enforces an authentication sequence using a one-time verification password to confirm user session integrity.
 - Maintains persistent, authenticated session records and supplies an explicit logout handler.
- **Module 2: Structured Intake & Ingestion Pipeline**

- **Lost Item Subsystem:** Captures item name, functional description, optional reference picture, contact telephone number, and student registration number.
 - **Found Item Subsystem:** Captures item title, condition details, mandatory photographic proof, exact location found, timestamp, and discoverer contact details.
 - Performs server-side validation and sanitizes uploaded media payloads into designated storage directories.
- **Module 3: Visual Feed Catalog & Peer Verification**
 - Displays active items inside an Amazon-style multi-column card gallery with optimized media thumbnails.
 - Conceals discoverer and owner contact information behind an explicit user-driven trigger to prevent exposure.
 - Directs claimants to verify ownership through peer-to-peer validation (passcodes, device pairing, or physical matching) prior to item transfer.
- **Module 4: Intelligent Semantic Matching Engine**
 - Integrates Google Gemini 2.5 Flash to automatically interpret submitted lost item text descriptions.
 - Compares newly logged lost queries against active found records to locate conceptual and semantic matches.
 - Renders a highlighted AI recommendation banner when a high-probability match is identified.

5. Non-Functional Requirements

- **NFR 1 - Security:** Strict domain-level validation filters unauthorized external traffic; all user input fields and file payloads are sanitized against directory traversal and script injection attacks.
- **NFR 2 - Usability:** The visual interface mirrors the design conventions and palette of the VIT Bhopal Student Portal (Navy #0A2540, Gold #F4B400, Blue #1E88E5), ensuring rapid onboarding and visual accessibility.
- **NFR 3 - Performance:** Grid layouts render synchronously with minimal overhead; binary media is stored on local disks with lightweight path references persisted in database tables.
- **NFR 4 - Reliability & Maintainability:** The backend leverages a modular architecture dividing business routing, database transactions, and client views into isolated files.

6. System Architecture

- **Presentation Layer:** Client-side HTML5 views styled with custom CSS and enhanced with vanilla JavaScript to handle form interactions, image previews, and UI visibility states.
- **Application Layer:** Python Flask server functioning as a controller that processes route transitions, session tokens, media uploads, and business logic.
- **Data Layer:** An embedded SQLite relational database (`items.db`) managing relational tables, constraints, and audit timestamps.
- **External AI Service Layer:** Google GenAI SDK interfacing asynchronously with `gemini-2.5-flash` to execute natural language text comparisons.

7. Design Diagrams (Structural & Behavioral Descriptions)

7.1 Use Case Diagram

- **Actors:** Enrolled Student, Finder, Claimant, Gemini AI Service.
- **Student Use Cases:** Authenticate with College ID, View Dashboard, Search Catalog Feed, Trigger Contact Reveal.
- **Finder Use Cases:** Access Found Form, Upload Image, Log Location & Time Metadata, Release Contact Details.
- **Claimant Use Cases:** Access Lost Form, Log Lost Query, View AI Recommendations, Validate Item Claim.

7.2 Workflow / Process Flow Diagram

- **Start:** User accesses landing page and inputs @vitbhopal.ac.in email.
- **Branch 1:** System verifies domain format; if invalid, display error; if valid, request OTP verification.
- **Branch 2:** User accesses Dashboard containing a 50/50 vertical division between "Lost" and "Found" sections.
- **Branch 3 (Lost Flow):** User fills Lost form → Data persists to DB → Gemini executes query scan → User is redirected to the catalog feed with AI recommendations.
- **Branch 4 (Found Flow):** User fills Found form with mandatory photo → Data persists to DB → User is redirected to the catalog feed.

7.3 Sequence Diagram

- **Step 1:** User client posts form payload to Flask </lost> endpoint.
- **Step 2:** Flask controller validates file input, secures the file name, and writes the asset to the static folder.
- **Step 3:** Flask executes an **INSERT** statement into the SQLite [lost_items](#) table.
- **Step 4:** Flask queries active records from the [found_items](#) table and formats a comparative prompt.
- **Step 5:** Flask dispatches the payload to the Gemini API endpoint and awaits candidate responses.
- **Step 6:** Flask renders [grid.html](#) with catalog cards and appends the AI recommendation string.

7.4 Class / Component Diagram

- **UserController Component:** Handles login verification, OTP sessions, and authentication state transitions.
- **ItemController Component:** Orchestrates CRUD operations across lost and found models.
- **StorageManager Component:** Handles multipart file streams, storage path mapping, and asset validations.
- **AIEngine Component:** Manages SDK client configuration, prompt structures, and response serialization.

7.5 Entity Relationship (ER) Diagram

- **User Entity:** `id` (PK, Integer), `email` (Text, Unique), `password_hash` (Text), `created_at` (Timestamp).
- **LostItem Entity:** `id` (PK, Integer), `name` (Text), `description` (Text), `contact` (Text), `student_id` (Text), `image` (Text, Nullable), `status` (Text), `created_at` (Timestamp).
- **FoundItem Entity:** `id` (PK, Integer), `name` (Text), `description` (Text), `found_location` (Text), `found_time` (Text), `contact` (Text), `image` (Text, Not Null), `status` (Text), `created_at` (Timestamp).

8. Design Decisions & Rationale

- **Decision 1 - Eliminating Formal Security Counters:** Since VIT Bhopal lacks an active campus lost desk, the platform uses direct peer communication, shifting physical verification to the handover interaction.
- **Decision 2 - Domain-Level Email Whitelisting:** Using regex email verification provides verification without requiring direct access to university database backends.
- **Decision 3 - Python Flask & SQLite Foundation:** Chosen over heavy frameworks to provide lightweight execution, straightforward file-based database backups, and clean code maintainability.
- **Decision 4 - Split Screen Dashboard Layout:** Eliminates complex multi-nested menus by presenting two equal, clear options immediately upon entry.

9. Implementation Details

- **Routing Architecture:** Standardized RESTful endpoints (`/`, `/dashboard`, `/lost`, `/found`, `/grid/<category>`, `/logout`) structured using Flask route decorators.
- **Database Driver:** Native Python `sqlite3` driver managing atomic transactions with parameterized SQL statements to eliminate injection vulnerabilities.
- **Media Processing:** Leveraged `werkzeug.utils.secure_filename` to prevent arbitrary filesystem writes during multipart file ingestion.
- **AI Orchestration:** Integrated `google-genai` SDK using `gemini-2.5-flash` for low-latency semantic matching over database record strings.

10. Screenshots & Results

- Note: Paste actual project screenshots into these corresponding sections in Google Docs:
 - **Figure 1:** University Portal Themed Login Screen with Email Validation Check.
 - **Figure 2:** Fifty-Fifty Split Screen Dashboard ("Lost" vs. "Found" Interactive Halves).
 - **Figure 3:** Lost Item Data Entry Interface.
 - **Figure 4:** Found Item Submission Interface with Mandatory File Upload.
 - **Figure 5:** Amazon-Style Card Catalog Displaying Active campus listings.
 - **Figure 6:** Contact Reveal Mechanism and Gemini AI Match Recommendation Banner.

11. Testing Approach

- **Unit Testing:** Verified regular expression matching routines for valid university email formats versus unauthorized public domains.
- **Functional Ingestion Testing:** Executed edge-case testing on forms (e.g., verifying that omitting mandatory images on the Found Form blocks submission).
- **Media Handling Testing:** Uploaded varied image formats (.jpg, .png, .webp) to verify file integrity and thumbnail generation.
- **Integration Testing:** Conducted end-to-end user tests from authentication through submission to confirm catalog updates.
- **AI Failure Handling Testing:** Verified system behavior when the GEMINI_API_KEY is missing or times out to ensure catalog browsing remains functional.

12. Challenges Faced

- **Challenge 1:** Managing item recovery lifecycles without an institutional database or central security desk.
 - *Resolution:* Built an on-demand contact reveal pattern enabling students to communicate and verify ownership directly.
- **Challenge 2:** Handling image uploads cleanly on low-resource environments.
 - *Resolution:* Standardized directory structures using unique timestamps and secure filenames while persisting relative path references in SQLite.
- **Challenge 3:** Accurately matching lost reports to found items despite different wording and descriptions.
 - *Resolution:* Integrated gemini-2.5-flash to evaluate the semantic meaning of descriptions rather than relying solely on exact keyword searches.

13. Learnings & Key Takeaways

- Developed practical skills in architecting full-stack web applications using Python, SQLite, and modern CSS layouts.
- Gained experience integrating modern Large Language Model APIs into transactional web applications.
- Learned how to translate real-world campus operational problems into clear technical requirements, relational database designs, and user workflows.

14. Future Enhancements

- **Automated Image Feature Extraction:** Using Gemini Vision to auto-populate category, color, and object tags directly from uploaded found photos.
- **Real-Time Notification Relays:** Implementing automated webhooks or Telegram bot alerts to notify students the moment an item matching their description is logged.
- **Automated Expiry Timers:** Setting item records to automatically archive after a defined timeframe (e.g., 30 days) to prevent feed clutter.

15. References

- Flask Web Framework Documentation: <https://flask.palletsprojects.com/>
- Google GenAI SDK & Gemini API Documentation: <https://ai.google.dev/docs>
- SQLite Documentation & SQL Reference: <https://www.sqlite.org/docs.html>
- MDN Web Docs for HTML5 Form Controls and Responsive CSS Grid Layouts

- VIT Bhopal Official Portal Color Palette and Brand Guidelines